Ejercicios de Práctica: Variables en JavaScript

Nivel 1 – Básicos

1. Declara una variable `pais` con tu país de nacimiento y muéstrala en consola.
   let pais = “España”;

   console.log(pais);
2. Declara una constante `velocidadLuz` con el valor `299792458`. Intenta cambiarle el valor y anota qué ocurre.
   Indica “TypeError” por asignar a una constante.
3. Declara una variable `numero1` con valor `10` y una variable `numero2` con valor `5`.
   - Muestra en consola su suma.
     var numero1 = 10;
     var numero2 = 5;

     console.log(numero1 + numero2);

     salida: 15
   - Luego cambia `numero1` a `20` y repite la suma.
     numero1 = 20;

     console.log(numero1 + numero2);

     salida: 25
4. Declara una variable sin asignarle valor. Imprime su contenido y explica el resultado.
   Indica “undefined” por pantalla. “Undefined” es un tipo que indica que la variable no se ha inicializado.

Nivel 2 – Uso de `let`, `const` y `var`

5. Crea un bloque `if (true) { ... }` en tu programa. Dentro declara:
   - Una variable `a` con `var`.
   - Una variable `b` con `let`.
   - Una constante `c` con `const`.

- Luego intenta imprimir las tres fuera del bloque. ¿Qué ocurre?
  La consola indica “ReferenceError”, aclarando que b no está definida.

6. Escribe un pequeño script que demuestre la diferencia entre **reasignar** (`let`) y **redeclarar** (`var`).

7. Declara una constante `colores` con un array de tres colores. Agrega un nuevo color al array y comenta por qué es posible modificarlo aunque sea una constante.
   const colores = ["rojo", "verde", "azul"];

   colores.push("blanco");

   console.log(colores);

   Se puede modificar porque no estamos reasignando el valor dentro de la constante, sino añadiendo.

Nivel 3 – Hoisting y valores por defecto

8. Escribe un programa que intente mostrar en consola el valor de una variable declarada con `var` antes de su declaración. ¿Qué valor aparece?
   Aparece “undefined”, indicando que la variable se ha iniciado, pero no se ha asignado nada. Ya que “var” se declara al inicio, pero no se inicializa hasta la orden que la inicializa.
9. Haz lo mismo con `let` y `const`. ¿Qué error aparece?
   Indica “ReferenceError”, especificando que no se puede acceder a “b” antes de inicializarlo.
10. Declara una variable con `let` sin asignarle valor, imprímela y reflexiona sobre el significado de `undefined`.
    En este caso, “undefined” indica que la variable no se ha inicializado, siendo “undefined” un tipo de JavaScript.

Nivel 4 – Aplicación práctica

11. Declara variables para guardar la información de un estudiante:
    - `nombre` (cadena),
    - `edad` (número),
    - `curso` (cadena),
    - `notaMedia` (número).
    - Muestra una frase con toda la información en la consola.

```
let nombre = "Iván";

let edad = 19;

let curso = "2DAM";

let notaMedia = "6.25";

console.log(

   "Nombre: " + nombre + ", edad: " + edad

   + ", curso: " + curso + ", nota media: "
```

```
    + notaMedia
);
```

12. Imagina que estás programando una calculadora de descuentos:
    - Declara con `const` una variable `precioOriginal`.
    - Declara con `let` una variable `descuento`.
    - Calcula el precio final restando el descuento al precio original y muéstralo en consola.

```
const precioOriginal = 20;
let descuento = 0.20;

var precioFinal = precioOriginal - (precioOriginal * descuento);

console.log(precioFinal);
```

13. Declara una constante llamada `IVA` con valor `0.21`.
    Luego crea una variable `precioSinIVA` con un valor numérico cualquiera.
    Calcula el precio final con IVA y muéstralo en consola.

```
const IVA = 0.21;
let precioSinIVA = 100;

const precioFinal = precioSinIVA - (precioSinIVA * IVA);

console.log("Precio final con IVA: " + precioFinal);
```

Nivel 5 – Retos

14. Declara una constante `persona` como objeto con propiedades `nombre` y `edad`.
    - Cambia el valor de `edad`.

- Intenta reasignar `persona` a otro objeto. Explica qué ocurre.

```
const persona = {
    nombre : "Ivan",
    edad : 19
};
console.log(persona);

persona.edad = 20;
console.log(persona);

persona = {
    nombre : "Hugo",
    apellido : "Perez"
};
```

La consola indica “TypeError”, aclarando que no puedes reasignar a una constante.

15. Crea un script donde declares una variable global con `var` y otra global con `let`.

- Intenta acceder a ambas desde dentro de una función y luego desde fuera.

```
var a = "var";
let b = "let";

function func1(){
    console.log("Dentro:");
    console.log(a);
    console.log(b);
}

func1();

console.log("Fuera:");
console.log(a);
console.log(b);
```

- Explica las diferencias de ámbito.
  En este ejemplo, como ambas variables son globales, son accesibles desde cualquier sitio dentro del script. Sin embargo, si fuesen declaradas dentro de la función, no serían accesibles desde fuera.

16. Simula un pequeño inventario:

- Declara `const nombreProducto = "Portátil"`.
- Declara `let stock = 15`.
- Resta 1 al stock para simular una venta.

- Muestra el mensaje: `"Quedan X unidades del producto Y"`.

```
const nombreProducto = "Portátil";
let stock = 15;
stock -= 1;

let mensaje = (`Quedan ${stock} unidades del producto ${nombreProducto}`)
console.log(mensaje);
```

Errores Típicos con Variables en JavaScript

1. Olvidar declarar la variable con `let` o `const`

Si no se usa `let` o `const`, JavaScript crea una variable **global implícita**, lo que puede generar errores graves y difíciles de rastrear.

```
ciudad = "Madrid"; // mala práctica (global implícita)
let ciudad = "Madrid"; // correcto
```

---

2. Confundir reasignar con redeclarar

- `let` y `const` **no permiten redeclaración** en el mismo bloque.
- `var` sí lo permite, lo que puede llevar a confusión.

```
let edad = 20;
// let edad = 25; Error: ya declarada en este bloque
edad = 25; // Reasignación permitida

var nombre = "Ana";
var nombre = "Luis"; // ✔ permitido, pero confuso
```

---

3. Creer que `const` significa “inmutable”

- `const` impide **reasignar la referencia**, pero **no bloquea cambios internos** en objetos o arrays.

```
const lista = [1, 2, 3];
lista.push(4); // permitido
// lista = [1, 2, 3, 4]; Error: no se puede reasignar
```

---

4. Problemas con el ámbito (scope)

- Variables declaradas con `var` se “escapan” del bloque `{}`.
- `let` y `const` sí respetan el ámbito de bloque.

```
if (true) {
    var x = 10;
    let y = 20;
}
console.log(x); // 10
// console.log(y); // Error: y no está definida
```

---

5. No entender el hoisting

- Con `var`, la declaración se eleva, pero no la inicialización.
- Con `let` y `const`, existe la "zona muerta temporal" (TDZ) que produce error si accedemos antes de la inicialización.

```
console.log(a); // undefined (por hoisting)
var a = 5;

// console.log(b); // Error
let b = 10;
```

---

6. No inicializar variables

Si se declara con `let` pero no se asigna valor, su contenido será `undefined`.
Esto es normal, pero puede confundir si no se entiende.

```
let dato;
console.log(dato); // undefined
```

---

7. Usar nombres poco descriptivos

Variables como `x1`, `dato`, `cosas` dificultan la comprensión del programa.
Mejor usar nombres significativos:

```
let totalCompra = 45; // claro
let t = 45; // poco descriptivo
```

---

8. Creer que el valor de una constante puede calcularse después

Muchos alumnos intentan declarar una `const` sin inicializarla, lo cual no es válido.

```
// const PI; Error: debe inicializarse
const PI = 3.1416; // correcto
```

---

9. Duplicar variables globales en distintos archivos

En proyectos grandes, si se usa `var` o variables globales sin control, se pueden **sobrescribir** entre sí.
Por eso se recomienda siempre trabajar con `let`/`const` y modularizar el código.

---

10. Creer que `let` y `const` se comportan igual que en otros lenguajes

Muchos alumnos vienen de C, Java o Python y esperan comportamientos idénticos.
Ejemplo: en Java o C# una constante **sí es completamente inmutable**, en JS **no siempre**.